



Security Audit

Superset (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	27
Conclusion	28
Our Methodology	29
Disclaimers	31
About Hashlock	32

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Superset team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Superset Finance is a next-generation liquidity infrastructure platform designed to address the fragmented state of stablecoin liquidity across multiple blockchains. It enables token issuers to deploy and configure stablecoins on over 100 chains with just one click, offering instant settlement and synchronized compliance. By combining liquidity across chains and incentivizing liquidity providers through swap-fee mining, Superset creates deep, efficient pools from day one. Aimed at simplifying multi-chain operations, it helps developers and issuers expand seamlessly without sacrificing liquidity.

Project Name: Superset

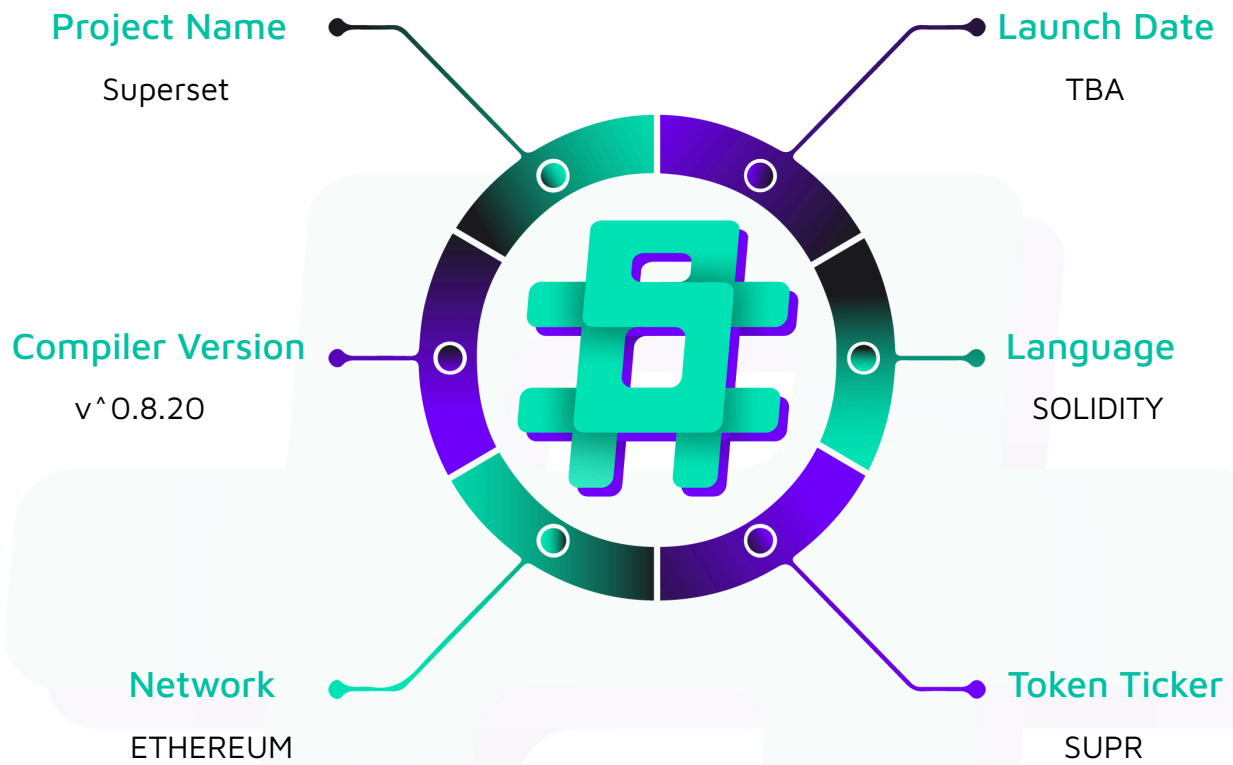
Project Type: DeFi

Compiler Version: ^0.8.20

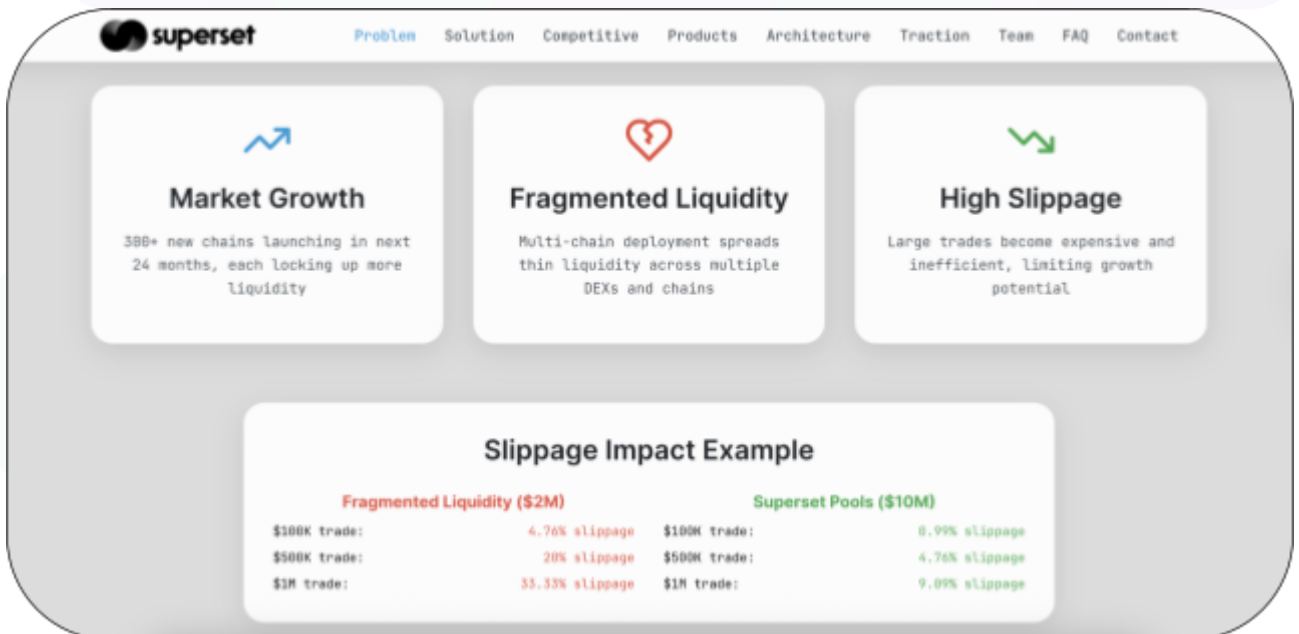
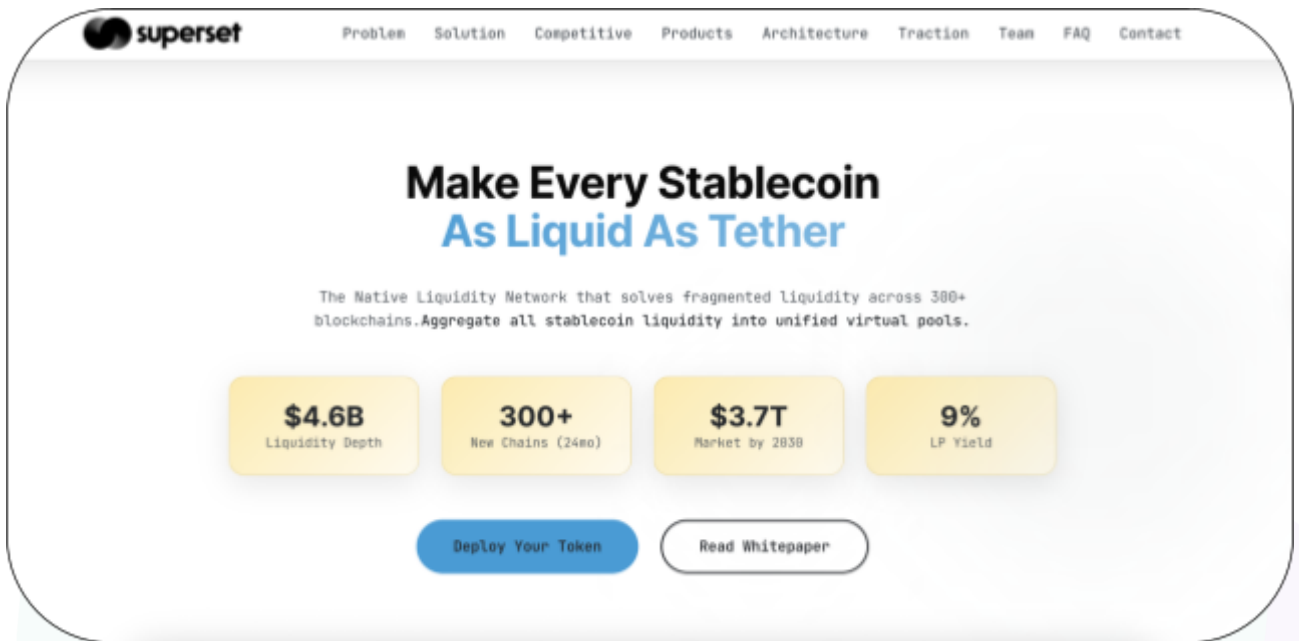
Website: <https://superset.finance/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Superset project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Superset Smart Contracts
Platform	Ethereum / Solidity
Audit Date	July, 2025
Contract 1	OAPPFactory.sol
Contract 2	OFTFactoryV2.sol
Contract 3	OFTFactoryV3.sol
Contract 4	OFTFactoryV3Lib.sol
Audited GitHub Commit Hash 1 (OAPP)	67f9986616e203d42a52f74cdd32706f081ad8d5
Audited GitHub Commit Hash 2 (OFT)	cdee97af70f669e725c0fb20f17f0e01c431bb2c
Fix Review GitHub Commit Hash 1	50eaacbbd19bca8d03efb9a49718534a48282741
Fix Review GitHub Commit Hash 2	d77abc33dfbe7f428d6206b85079c23abd741d4f

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 Medium severity vulnerabilities

2 Low severity vulnerabilities

5 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
OAPPFactory.sol - Abstract factory for deploying and configuring OAPP contracts - Deterministic CREATE2 deployment and initial configuration - Manages peer relationships, enforces LayerZero options, and ULN config - Supports single-chain and multi-chain create/configure flows (createOappMulti, configureMulti) - Manages admins, bytecode whitelist, fee exemptions, and deployment fees	Contract achieves this functionality.
OFTFactoryV2.sol - Factory for deploying and configuring OFT and OFTAdapter contracts on EVM - Deterministic CREATE2 deployment of MyOFT via _createOFT - Deterministic CREATE2 deployment of OFTAdapter via _createOFTAdapter - Handles cross-chain create/configure messages in _lzReceive override - Emits OFTCreated, OFTAdapterCreated, and LzMessageReceived events	Contract achieves this functionality.
OFTFactoryV3.sol - Extends OFTFactoryV2 with EVM↔Solana interoperability - Integrates upgradable OFTFactoryV3Lib for encoding, decoding, and security checks	Contract achieves this functionality.

- Overrides <code>_lzReceive</code> to process Solana-origin messages via the library - Provides <code>solanaQuoteRemoteCreateMulti</code>, <code>solanaCreateOappMulti</code>, and <code>Solana-aware configureMulti</code> entry points	
OFTFactoryV3Lib.sol - Library for RLP decoding and validation of Solana-origin OFT messages - <code>rlpDecode*</code> functions decode seeds, peers, create/configure payloads - <code>processLzReceive</code> normalizes messages into <code>LzReceiveData</code> format - <code>createSolanaOapp</code> and <code>handleSolanaConfigureBatch</code> build secure <code>LzSendParam</code> arrays - Enforces gas/value thresholds and salt/peer consistency for message batches	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Superset project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation; however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Superset project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] OFTFactoryV2/OFTFactoryV3#_lzReceive - Same addresses might not belong to the same users on different chains due to smart wallet usage

Description

The `OFTFactoryV2` and `OFTFactoryV3` contracts allow a user to specify an admin address in the cross-chain payload, but never verify that the admin belongs to the original sender on the destination chain.

Vulnerability Details

When a `CREATE` message arrives, `_lzReceive` decodes `CreateOFTData` and calls `_createOFT` with the supplied `oftData.admin`, then in `_deployAndConfigure` it sets `admins[contractAddress] = admin` without any proof:

```
IOFTFactory.CreateOFTData    memory    oftData    =    abi.decode(dataReceived.data,
(IOFTFactory.CreateOFTData));

_createOFT(..., oftData.admin, ..., sender);

// ...

admins[contractAddress] = admin;
```

Because `CREATE2` yields the same wallet address `W` on every chain, an attacker can frontrun the victim's message by deploying a malicious wallet at `W` on `Chain B` before her message is processed. The factory then trusts the attacker's wallet as the admin of the victim's newly created OFT.

Scenario:

A. Alice deploys a smart wallet at address `W` on `Chain A` using `CREATE2` with deterministic parameters as per her smart wallet provider factory.

B. Alice calls `remoteCreate()` to create an OFT on Chain B, specifying her wallet address W as the admin in `CreateOFTData`

C. Bob monitors pending LayerZero messages and observes Alice's creation request

D. Bob frontruns the message by deploying a malicious contract at the same address W on Chain B using the same `CREATE2` factory and salt

E. When Alice's cross-chain message arrives and is processed by `_lzReceive()`, the admin is set to address W

F. Since Bob now controls the contract at address W on Chain B, he becomes the admin of Alice's newly created OFT

Bob can now call `configureOAPP()`, `transferOwnership()`, and perform other admin actions to steal funds or manipulate the OFT

Impact

An attacker who controls the admin address can call `configureOAPP`, `transferOwnership`, `mint` or `burn` tokens, and drain funds. Alice has no way to recover control or assets once Bob's contract is accepted as admin.

Recommendation

Require the sender to sign the payload on the source chain and in `_lzReceive` verify that signature with `ECDSA.recover` matches the admin field before calling `_createOFT` or setting admins. This proof ensures that only the legitimate owner of the private key can become an admin.

Note

Superset's team informed Hashlock that the issues have been acknowledged and that the chosen mitigation involves warning users through the UI, without implementing protocol-level changes.

Status

Acknowledged

[M-02] OFTFactoryV3#configureOAPP - Account Abstraction transactions can bypass admin auth

Description

The OFTFactoryV3 contract checks admin privileges by comparing `msg.sender` against its `admins` mapping.

In an Account Abstraction environment, when the users interact with the protocol with their AA wallets, every on-chain call is relayed through a `forwarder` contract, so `msg.sender` becomes the `forwarder`, not the user's AA wallet.

This makes any authorization built on `if (msg.sender != admins[oapp]) revert OnlyAdmin();` completely ineffective, because any relayer can pose as the admin.

Vulnerability Details

When a user submits a transaction through a trusted forwarder, the forwarder's address appears as `msg.sender`, and the user's wallet address is embedded in the call data. All of OFTFactoryV3's admin gated functions, including `configureOAPP`, `createOFT`, `createOFTAdapter`, `createOappMulti`, `solanaCreateOappMulti`, and `configureMulti`, use a simple `msg.sender` check.

For example:

```
function configureOAPP(
    address OAPP,
    IOAPPFactory.Peer[] calldata peers
) public {
    if (msg.sender != admins[OAPP]) revert OnlyAdmin();
    _configureOAPP(OAPP, peers);
    emit ConfiguredOAPP(OAPP, peers);
}
```

In this scenario, a malicious relayer can invoke `configureOAPP` via the `forwarder` and satisfy the `msg.sender == admins[OAPP]` check by having the admin mapping point to



the forwarder. The contract never unpacks the real user from the call data, so the attacker fully bypasses admin controls.

Impact

Any relayer can call privileged factory functions and take over OAPP contracts. The attacker can add or remove peers, transfer ownership, update configuration, withdraw funds, or perform any admin-only operation. This completely undermines the intended access control and exposes all deployed OAPPs to takeover by unauthorized parties.

Recommendation

Eliminate reliance on `msg.sender` for admin checks and instead require a signature from the registered admin. Each admin gated function should accept a `bytes signature` parameter and verify it with `ECDSA.recover`.

For example:

```
import "@openzeppelin/contracts/utils/cryptography/ECDSA.sol";

function configureOAPP(
    address OAPP,
    IOAPPFactory.Peer[] calldata peers,
    bytes calldata adminSig
) public {
    bytes32 payloadHash = keccak256(
        abi.encodePacked(
            address(this),
            "configureOAPP",
            OAPP,
            peers
        )
    );
    address signer = ECDSA.recover(ECDSA.toEthSignedMessageHash(payloadHash),
adminSig);
    require(signer == admins[OAPP], "Invalid admin signature");
    _configureOAPP(OAPP, peers);
    emit ConfiguredOAPP(OAPP, peers);
}
```

With this change, only the private key holder of `admins[0APP]` can produce a valid signature, and no `forwarder` or relayer can impersonate them.

Note

Superset's team informed Hashlock that the issues have been acknowledged and that the chosen mitigation involves warning users through the UI, without implementing protocol-level changes.

Status

Acknowledged

Low

[L-01] OAPFactory#payNative/_checkFee - Extra ETH not refunded

Description

In both the `_payNative` and `_checkFee` functions, the contract only enforces that the caller sends at least the required fee and then forwards that exact amount to the `owner`. Any surplus `msg.value` above what's needed is neither checked nor returned, so it remains locked in the contract or immediately goes to the `owner` without ever reaching the sender again.

Users who accidentally overpay by including extra ETH with their transaction will lose those funds.

Recommendation

After sending the required fee, calculate `refund = msg.value - requiredFee` and explicitly send refund back to `msg.sender`

Status

Acknowledged

[L-02] OUTFactoryV3Lib#rlpDecodeConfigureParam - Missing RLP array length check

Description

In `rlpDecodeConfigureParam`, the code does:

```
RLPReader.RLPItem[] memory ls = rlpBytes.toRlpItem().toList();  
//@audit no check on ls.length  
result.OAPP = ls[0].toAddress();  
senderBytes = bytes32(ls[2].toUint());  
...
```

Because there is no `if (ls.length != 3) revert` check, a malformed RLP payload with fewer than three elements will cause an out-of-bounds array and an unexpected revert.

Recommendation

Add a length check at the start of the function.

Status

Resolved

QA

[Q-01] **OFTFactoryV3#solanaCreateOappMulti** - External calls in `_executeLzSends` pose no reentrancy risk

Description

In `solanaCreateOappMulti`, the line;

```
function solanaCreateOappMulti(
    bytes calldata localCreateMessage,
    IOAPPFactory.RemoteCreateParam[] calldata remoteCreateMessagesSolana,
    IOAPPFactory.RemoteCreateParam[] calldata remoteCreateMessages
) external payable returns (MessagingReceipt[] memory msgReceipts) {
    IOFTFactoryV3Lib.LzSendParam[] memory params =
    oftFactoryV3Lib.createSolanaOapp(address(this), remoteCreateMessagesSolana,
    msg.sender);
    >>> _executeLzSends(params); // @audit
    _checkFee();
    return createOappMulti(localCreateMessage, remoteCreateMessages);
}
```

The `_executeLzSends` iterates over a batch of messages and calls the internal `_lzSend` helper, which under the hood invokes the LayerZero endpoint contract's `send` method. This could trigger the fallback mechanism of the receiving address (provided that it's a contract) when the surplus fee is returned and reenter the contract.

Recommendation

We recommend adding a `nonReentrant` modifier for future upgrades.

Status

Resolved

[Q-02] IOFTFactoryV3Lib#rlpDecodeOFTData - Store opcode 0x00 decoded as CREATE

Description

In the RLP decoding function `rlpDecodeOFTData`, the first byte of a Solana origin message is called the operation. Although `ACTION_STORE` is defined as `0x00`, the code immediately routes any `0x00` opcode into the create path and sets `SendOperation.CREATE`:

```
function rlpDecodeOFTData(bytes memory data) public pure returns (bytes memory) {
    uint8 operation = uint8(data[0]);           // operation == 0x00 (ACTION_STORE)
    bytes memory rlpBytes = removeFirstByte(data);
    if (operation == 0x00) {                     // ACTION_STORE payload
        (IOFTFactory.CreateOFTData memory result, bytes32 senderBytes) =
        rlpDecodeCreateOFTData(rlpBytes);
        return abi.encode(IOAPPFactory.LzReceiveData({
            senderOrOapp: senderBytes,
            data: abi.encode(IOFTFactory.CreateData({ isAdapter: false, data:
            abi.encode(result) })),
            operation: IOAPPFactory.SendOperation.CREATE // always CREATE
        })));
    } else if (operation == 0x01) {
        ...
    } else {
        revert InvalidInput();
    }
}
```

When `processLzReceive` handles a Solana message, it calls this function unconditionally for EIDs `30168` and `40168`, causing every store opcode to be treated as a creation request:

```
function processLzReceive(bytes calldata _message, uint32 srcEid) external pure returns
(LzReceiveResult memory result) {
    if (srcEid == 30168 || srcEid == 40168) {
        convertedMessage = rlpDecodeOFTData(_message); // store op maps to CREATE
    } else {
```

```
        convertedMessage = _message;
    }
    IOAPPFactory.LzReceiveData memory dataReceived = abi.decode(convertedMessage,
        (IOAPPFactory.LzReceiveData));
    ...
    if (dataReceived.operation == IOAPPFactory.SendOperation.CREATE) { ... }
}
```

This does not currently break the deployed logic, but can mislead anyone reading the code or transaction logs, since ACTION_CREATE is formally 0x01. Treating 0x00 as a create action may obscure the intended semantics of a store or update operation versus a full create.

Recommendation

Either introduce a distinct STORE variant in the SendOperation enum for opcode 0x00 or rename ACTION_STORE to something like ACTION_STORE_CREATE to clarify that 0x00 messages are decoded into create flows on EVM.

Status

Resolved

[Q-03] OFTFactoryV3Lib#rlpDecodeOFTData - CREATE (0x01) treated as CONFIGURE; true CONFIGURE/METADATA (0x02/0x03) always revert

Description

Inside `rlpDecodeOFTData`, the first byte of incoming Solana messages is interpreted as an operation, but only two cases are handled:

```
uint8 operation = uint8(data[0]);
bytes memory rlpBytes = removeFirstByte(data);

if (operation == 0x00) {
    // Decode as a CreateOFTData and emit SendOperation.CREATE
} else if (operation == 0x01) {
    // Decode as a ConfigureParam and emit SendOperation.CONFIGURE
} else {
    revert InvalidInput(); //@audit 0x02 and 0x03 are not recognized
}
```

Even though `0x01` is the designated CREATE opcode in the Solana side convention, it is routed into the configure branch here. Any real configure (`0x02`) or metadata (`0x03`) payloads immediately hit `InvalidInput()` and revert.

Recommendation

No urgent fix is needed if all Solana clients already wrap create calls in `0x00` store style messages.

Status

Resolved

[Q-04] OAPFactory#isBytecodeWhitelisted - Public view function missing return type

Description

The function `isBytecodeWhitelisted` is declared as `public view` but does not declare any return value in its signature, even though it effectively returns by not reverting:

```
function isBytecodeWhitelisted(address _address) public view { // no return type
    if (!bytecodeWhitelist[_address]) {
        revert InvalidBytecode();
    }
}
```

Because it has no return type, this check-only function cannot be called off-chain as a simple boolean query via the contract's ABI. Users or integrators cannot make a call to get true/false and must instead rely on catching a revert or not.

Recommendation

Add a boolean return to the function signature so it can be queried cleanly.

Status

Resolved

[Q-05] OFTFactoryV3Lib#validateLZReceiveOptions - Use constants to indicate the purpose of magic numbers

Description

In OFTFactoryV3Lib.sol, calls to pass the literal values 600000 and 18000000 directly. Because these magic numbers aren't named, it isn't clear what they represent (gas limits, message value thresholds, etc.), and any change to those requirements must be hunted down and updated in every call site.

```
validateLZReceiveOptions(param, 600000, 18000000);
```

Recommendation

Declare descriptive constants at the top of the contract, for example;

```
uint128 constant MIN_PEER_INIT_GAS    = 600_000;  
uint128 constant MIN_PEER_INIT_VALUE = 18_000_000;
```

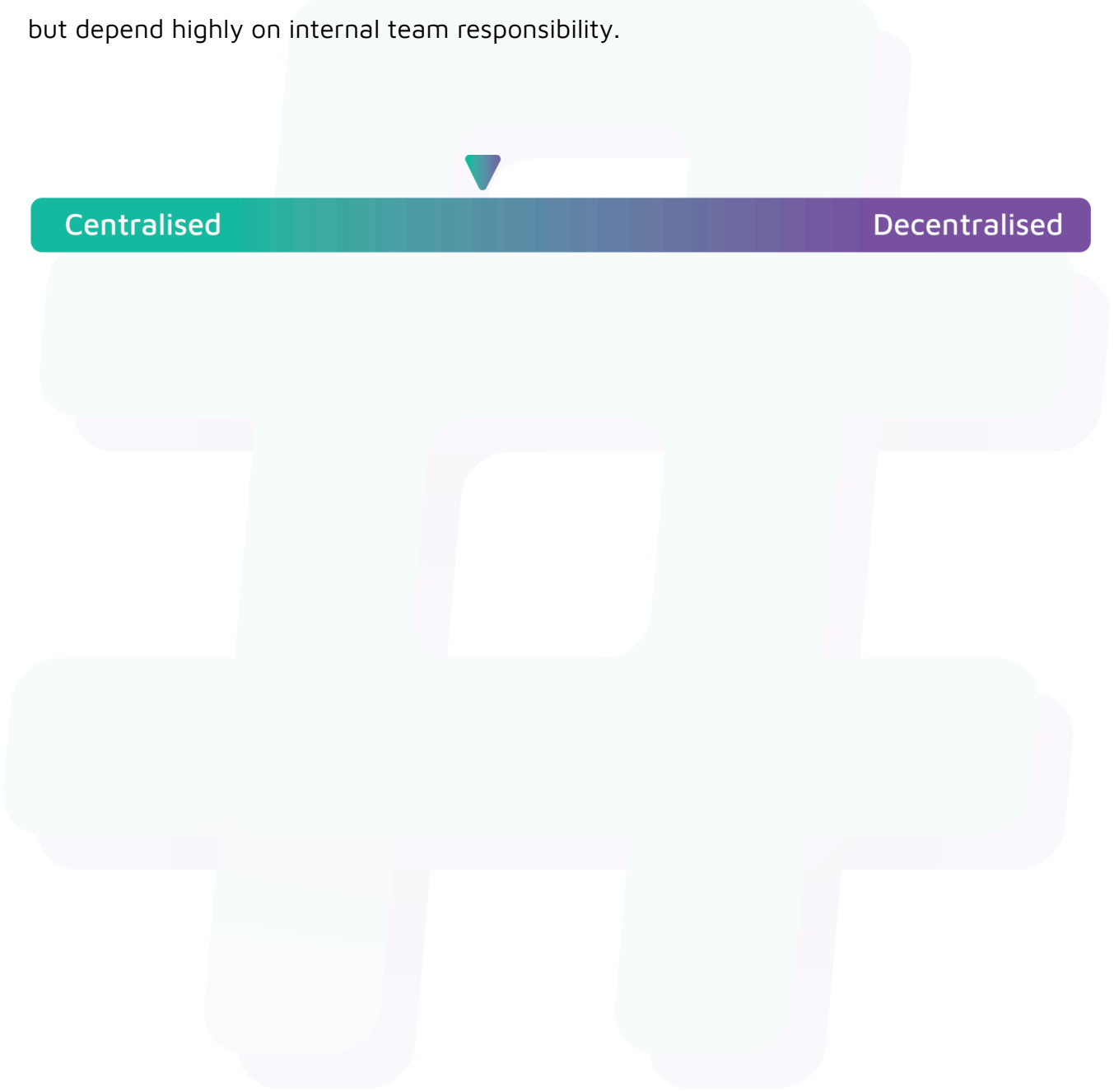
Status

Resolved

Centralisation

The Superset project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Superset project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.